Applications and Current Topics

Last modified: 2026-11-04

```
# The Python chunks on this page import these; reticulate provisions them.
# Attaching reticulate gives the inline R its `py` object.
library(reticulate)
source("../tools/use-torch.R")
use_torch(c("numpy", "pandas", "matplotlib"))
```

Brian Hutchinson's CSCI 481/581 sets the order of topics in these notes, which follow it with his permission. His course has no module on these topics, so this page is new, exercises included.

1 Welcome

1.1 Plan for this lecture

Earlier chapters built small networks and trained them in seconds. This lecture looks at what the same parts become at scale, and where they are used:

1. how many parameters[1] a language model built from Transformer blocks has, and how much computation training it takes;
2. how a pretrained model can do a new task from examples in its input alone;
3. how to adapt a large model by training a small number of new parameters;
4. how speech becomes an input that image networks can read;
5. how a Transformer can classify images.

1.2 The digits

```
import copy
import wave

import numpy as np
import pandas as pd
import torch
from torch import nn
import matplotlib.pyplot as plt

digits = pd.read_csv("data/optdigits-test.csv", header=None)
x = torch.tensor(digits.iloc[:, :64].to_numpy(), dtype=torch.float32) / 16
y = torch.tensor(digits[64].to_numpy())
order = torch.randperm(len(x), generator=torch.Generator().manual_seed(481))
train_idx, adapt_idx, val_idx = order[:1000], order[1000:1200], order[1200:]
n_train, n_adapt, n_val = len(train_idx), len(adapt_idx), len(val_idx)
loss_fn = nn.CrossEntropyLoss()


def fit(net, params, inputs, targets, n_steps=300):
```

[1] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

```
    # Full-batch Adam on the given parameters only.
    optimizer = torch.optim.Adam(params, lr=0.01)
    for step in range(n_steps):
        optimizer.zero_grad()
        loss_fn(net(inputs), targets).backward()
        optimizer.step()


def accuracy(net, inputs, targets):
    with torch.no_grad():
        return (net(inputs).argmax(dim=1) == targets).float().mean().item()


print(n_train, n_adapt, n_val)
```

```
1000 200 597
```

One exercise and two examples use the 1,797 digits of `data/optdigits-test.csv`[2] [1], each an 8 by 8 image. 1,000 random images form a training set[3], another 200 are kept for adapting a trained network, and the other 597 form a validation set[4]. `fit()` trains the parameters[5] it is given by full-batch Adam[6], and `accuracy()` gives the share of images whose largest logit is the true digit.

2 Large language models

2.1 How many parameters

Exercise 2.1 (Counting GPT-2's parameters)**.** The smallest GPT-2 language model has a vocabulary of 50,257 tokens, a context of 1,024 positions, width[a] $d = 768$, and 12 Transformer blocks, each with 12 heads and an MLP whose hidden layer has $4d$ units [2], [3]. Every linear layer has a bias, each block has two layer normalizations, one more follows the last block, and each layer normalization holds $2d$ numbers. Its output layer reuses the token embedding as its weights [3]. How many parameters[b] does it have?

[a] https://morrison-lab.github.io/lds/chapters/neural-nets.html#def-width
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

Solution

Solution 2.1.

[2] https://morrison-lab.github.io/dds/chapters/data/optdigits-test.csv
[3] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set
[4] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-validation-set
[5] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[6] https://morrison-lab.github.io/mds/optimization-gradient-descent.html#def-adam

```
d_model, vocab_size, context, n_blocks = 768, 50_257, 1_024, 12
block = nn.ModuleList([
    nn.LayerNorm(d_model),
    nn.MultiheadAttention(d_model, num_heads=12),
    nn.LayerNorm(d_model),
    nn.Linear(d_model, 4 * d_model),
    nn.Linear(4 * d_model, d_model),
])
per_block = sum(p.numel() for p in block.parameters())
embeddings = vocab_size * d_model + context * d_model
final_norm = 2 * d_model
gpt2_params = embeddings + n_blocks * per_block + final_norm
block_weights = 12 * d_model**2
block_rest = per_block - block_weights
print(per_block, block_rest, embeddings, gpt2_params)
```

```
7087872 9984 39383808 124439808
```

Building one block from PyTorch's layers and counting its parameters[a] gives 7,087,872 per block. Of these parameters, the attention's four $d \times d$ weight matrices hold $4d^2$, and the MLP's two weight matrices hold $8d^2$, 7,077,888 in all. The biases and the two layer normalizations hold the other 9,984. The token and position embeddings hold 39,383,808, and a last layer normalization holds $2d$. In all, GPT-2 has 124,439,808 parameters, the 124 million that Hugging Face lists for it [2]. The output layer adds none, since it reuses the token embedding.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

Definition 2.1 (Large language model (LLM))**.** A **large language model**, or **LLM**, is a language model made of Transformer blocks, with far more parameters[a] than the networks of earlier chapters, pretrained to predict the next token of a large collection of text. No agreed size makes a language model large.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

Example 2.1 (GPT-3)**.** GPT-3 is a large language model with 175 billion parameters[a] [4, Sec. 2.1].

```
gpt3_ratio = 175e9 / gpt2_params
print(gpt3_ratio)
```

```
1406.3023948092236
```

That count is 1,406 times GPT-2's 124,439,808.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

2.2 The cost of training

Exercise 2.2 (The cost of training)**.** Training a language model with N parameters[a] on D tokens takes about $6ND$ floating-point operations, or FLOPs [5, Sec. 3.3]. Hoffmann et al. [5] trained Chinchilla, a model with 70 billion parameters, on 1.4 trillion tokens. How many FLOPs did that take?

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

Solution

Solution 2.2.

```
chinchilla_params, chinchilla_tokens = 70e9, 1.4e12
chinchilla_flops = 6 * chinchilla_params * chinchilla_tokens
tokens_per_param = chinchilla_tokens / chinchilla_params
print(f"{chinchilla_flops:.2e}", tokens_per_param)
```

```
5.88e+23 20.0
```

About 5.9×10^{23} FLOPs. Each token costs about 6 operations per parameter[a]: 2 for the multiply and add of the forward pass, and about 4 for the backward pass, which takes about twice the computation of the forward pass [6, Sec. 2.1]. Chinchilla saw 20 tokens for each of its parameters[b].

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

Remark (Spending a compute budget)

Remark 2.1 (Spending a compute budget). For a fixed number of FLOPs, a larger model sees fewer tokens. Hoffmann et al. [5] trained over 400 language models to find the best split, and found that the number of parameter[a]s and the number of tokens should grow equally: each doubling of model size should come with a doubling of the tokens. Chinchilla's 20 tokens per parameter follow that rule, and it outperformed Gopher, a model 4 times its size trained with the same compute on fewer tokens [5].

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

2.3 Tasks from the prompt

Exercise 2.3 (Translation without training)**.** A pretrained language model has learned only to continue text. Without changing any of its parameters[a], how could you get it to translate English words into French?

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

Solution

Solution 2.3. Write the start of a document in which translations are the natural next text, such as a few English words, each followed by an arrow and its French translation, and then one more English word followed by an arrow. A model that has learned to continue such lists from its training text will continue this one with a French word. The task is described only by the input.

Definition 2.2 (In-context learning)**. In-context learning** is a pretrained language model doing a task from a description of it, or a few examples of it, written into its input, with no change to its parameters[a] [4, Secs. 1, 2].

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

Example 2.2 (A few-shot prompt)**.** Here is a prompt in the few-shot format of Brown et al. [4, Fig. 2.1]:

```
Translate English to French:
cat => chat
house => maison
water => eau
bread =>
```

Brown et al. [4] call a prompt with no examples zero-shot, one with one example one-shot, and one with a few examples few-shot, so the three translated examples make this prompt few-shot in-context

learning. The model's continuation after the last arrow is its answer. The English and French words here are ours, not Brown and colleagues'.

3 Adapting a large model

3.1 A low-rank update

Exercise 3.1 (A low-rank update)**.** Fine-tuning a 768×768 weight matrix $\mathbf{W}$ trains all of its entries. Suppose instead that the update to $\mathbf{W}$ is the product $\mathbf{BA}$ of a 768×8 matrix $\mathbf{B}$ and an 8×768 matrix $\mathbf{A}$. How many numbers does training the update learn?

Solution

Solution 3.1.

```
full_count = 768 * 768
low_rank_count = 768 * 8 + 8 * 768
print(full_count, low_rank_count, low_rank_count / full_count)
```

```
589824 12288 0.020833333333333332
```

$\mathbf{B}$ and $\mathbf{A}$ hold 12,288 numbers between them, 2% of the 589,824 in $\mathbf{W}$. The product $\mathbf{BA}$ is still a 768×768 matrix, but its rank[a] is at most 8.

[a] https://morrison-lab.github.io/mds/linear-algebra-subspaces.html#def-rank

Definition 3.1 (Low-rank adaptation (LoRA))**. Low-rank adaptation**, or **LoRA**, adapts a pretrained layer with weight matrix $\mathbf{W}_0$ by keeping $\mathbf{W}_0$ fixed and learning an $m \times r$ matrix $\mathbf{B}$ and an $r \times n$ matrix $\mathbf{A}$, where $\mathbf{W}_0$ is $m \times n$ and the rank[a] r is much smaller than m and n. The adapted layer maps $\underset{\sim}{x}$ to $\mathbf{W}_0\underset{\sim}{x} + \mathbf{BA}\underset{\sim}{x}$ [7, Sec. 4.1]. $\mathbf{B}$ starts at zero, so the adapted network starts as the pretrained one. Hu et al. [7] also scale $\mathbf{BA}\underset{\sim}{x}$ by a constant.

[a] https://morrison-lab.github.io/mds/linear-algebra-subspaces.html#def-rank

Example 3.1 (Adapting to mirrored digits)**.**

```
class LoRALinear(nn.Module):
    # A frozen nn.Linear plus a trainable update B @ A of rank `rank`.
    def __init__(self, linear, rank):
        super().__init__()
        self.linear = linear
        self.A = nn.Parameter(torch.randn(rank, linear.in_features) * 0.01)
        self.B = nn.Parameter(torch.zeros(linear.out_features, rank))

    def forward(self, inputs):
        return self.linear(inputs) + inputs @ self.A.T @ self.B.T


_ = torch.manual_seed(0)
base = nn.Sequential(nn.Linear(64, 256), nn.ReLU(), nn.Linear(256, 10))
fit(base, base.parameters(), x[train_idx], y[train_idx])
mirrored = x.reshape(-1, 8, 8).flip(-1).reshape(-1, 64)
base_acc = accuracy(base, x[val_idx], y[val_idx])
base_mirrored_acc = accuracy(base, mirrored[val_idx], y[val_idx])

lora_net = copy.deepcopy(base)
_ = lora_net.requires_grad_(False)
lora_net[0] = LoRALinear(lora_net[0], rank=8)
lora_params = [p for p in lora_net.parameters() if p.requires_grad]
fit(lora_net, lora_params, mirrored[adapt_idx], y[adapt_idx])
lora_count = sum(p.numel() for p in lora_params)
lora_acc = accuracy(lora_net, mirrored[val_idx], y[val_idx])

full_net = copy.deepcopy(base)
fit(full_net, full_net.parameters(), mirrored[adapt_idx], y[adapt_idx])
full_count_digits = sum(p.numel() for p in full_net.parameters())
full_acc = accuracy(full_net, mirrored[val_idx], y[val_idx])

_ = torch.manual_seed(0)
fresh_net = nn.Sequential(nn.Linear(64, 256), nn.ReLU(), nn.Linear(256, 10))
fit(fresh_net, fresh_net.parameters(), mirrored[adapt_idx], y[adapt_idx])
fresh_acc = accuracy(fresh_net, mirrored[val_idx], y[val_idx])
print(base_acc, base_mirrored_acc)
```

```
0.9631490707397461 0.4422110617160797
```

```
print(lora_count, lora_acc, full_count_digits, full_acc, fresh_acc)
```

```
2560 0.8911222815513611 19210 0.9028475880622864 0.9195979833602905
```

```
# Stop the render if the prose below stops being true.
assert lora_acc - base_mirrored_acc > 0.3
assert abs(full_acc - lora_acc) < 0.05
assert fresh_acc > max(lora_acc, full_acc) - 0.02
```

`base` is an MLP with one hidden layer of 256 ReLU units, trained on the 1,000 training digits. It classifies 96% of the validation digits correctly, but only 44% of the same digits mirrored left to right. LoRA with rank[a] 8 on its first layer trains 2,560 numbers on 200 mirrored digits, and raises the accuracy[b] on the mirrored validation digits to 89%. Fine-tuning all 19,210 parameters[c] on the same images reaches 90%. A new network trained on the 200 mirrored digits alone reaches 92%, so in this run the first training gains this small network nothing. LoRA pays off where a pretrained model is too large to copy and train for each task: Hu et al. [7] report that, for GPT-3, it cuts the trainable parameters by a factor of 10,000.

[a] https://morrison-lab.github.io/mds/linear-algebra-subspaces.html#def-rank

[b] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy
[c] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

3.2 Following instructions

Remark (From predicting text to following instructions)

Remark 3.1 (From predicting text to following instructions). A pretrained language model continues text; it does not answer a question just because it is asked one. Ouyang et al. [8] fine-tune GPT-3 in two stages to follow instructions. First they train it on prompts paired with answers that people wrote. Then people rank several of the model's answers to each prompt, a second network learns to predict those rankings, and reinforcement learning[a] trains the model toward answers that the second network rates highly. The second stage is reinforcement learning from human feedback, or RLHF.

[a] https://morrison-lab.github.io/lds/chapters/anatomy.html#def-reinforcement

4 Speech

4.1 Sound as a picture

Exercise 4.1 (Sound for an image network)**.** A recording is a sequence of air-pressure readings, thousands of them a second. How could a network built from convolutional layer[a]s for images take a recording as its input?

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-convolutional-layer

Solution

Solution 4.1. Turn the recording into a picture of which frequencies are loud at each moment. Cut the recording into short, overlapping windows, and measure how much of each frequency each window holds. Laid side by side, one column per window and one row per frequency, the measurements form a two-dimensional array, which a network for images can read.

Definition 4.1 (Spectrogram)**.** The **spectrogram** of a recording is the array whose entry in row f and column t is the magnitude of frequency f in the tth window of the recording. The windows are short and overlap, and the magnitudes come from a Fourier transform of each window, the short-time Fourier transform.

Example 4.1 (A spoken seven)**.**

```
with wave.open("data/7_jackson_0.wav") as recording:
    sample_rate = recording.getframerate()
    samples = np.frombuffer(recording.readframes(recording.getnframes()), dtype=np.int16)
sound = torch.tensor(samples, dtype=torch.float32) / 32768
window = 256
spectrogram = torch.stft(sound, n_fft=window, hop_length=window // 4,
                         window=torch.hann_window(window), return_complex=True).abs()
seconds = len(sound) / sample_rate
n_samples = len(sound)
n_freqs, n_frames = spectrogram.shape
freqs = torch.linspace(0, sample_rate / 2, n_freqs)
cutoff = 1000
power = spectrogram**2
low_share = (power[freqs < cutoff].sum() / power.sum()).item()
print(sample_rate, len(sound), seconds, spectrogram.shape)
```

```
8000 3457 0.432125 torch.Size([129, 55])
```

```
fig, axes = plt.subplots(2, 1, figsize=(6, 4.5), sharex=True)
times = np.arange(len(sound)) / sample_rate
axes[0].plot(times, sound.numpy(), linewidth=0.5);
axes[0].set_ylabel("pressure");
axes[1].imshow(spectrogram.log1p().numpy(), origin="lower", aspect="auto",
               extent=[0, seconds, 0, sample_rate / 2], cmap="magma");
axes[1].set_xlabel("time (seconds)");
axes[1].set_ylabel("frequency (Hz)");
fig.tight_layout();
plt.show()
```

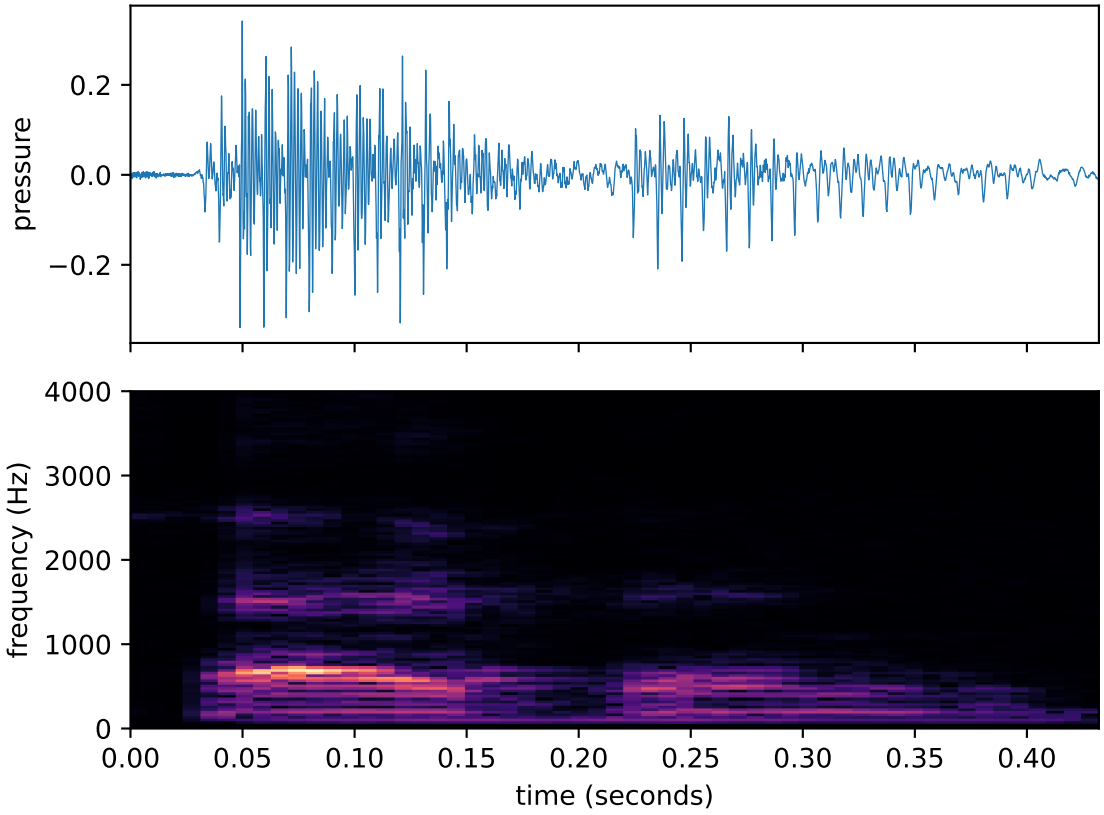


Figure 1: A recording of the word “seven” and its spectrogram, on a log scale.

`data/7_jackson_0.wav`[a] is one speaker saying “seven”, from the Free Spoken Digit Dataset [9]. It holds 3,457 readings at 8,000 a second, 0.43 seconds. Windows of 256 readings, each starting 64 readings after the one before, give a spectrogram with 129 frequency rows and 55 time columns. Frequencies below 1,000 hertz hold 93% of the recording’s power, in a few bright bands during the two vowels.

[a] https://morrison-lab.github.io/dds/chapters/data/7_jackson_0.wav

Remark (Spectrograms in speech recognition)

Remark 4.1 (Spectrograms in speech recognition). Whisper, a speech recognizer, turns each recording into a spectrogram on the mel scale, which spaces its rows more finely at low frequencies than at high ones, with 80 rows and one column every 10 milliseconds. An encoder of Transformer blocks reads the spectrogram, and a decoder writes out the text [10, Sec. 2.2].

5 Vision

5.1 Images as sequences

Exercise 5.1 (An image as a sequence)**.** A Transformer takes a sequence of vectors. Cut each 8 by 8 digit into 2 by 2 squares of pixels, and treat the 4 pixels of each square as one vector of the sequence. How many vectors does a digit become?

Solution

Solution 5.1.

```
patches = x.reshape(-1, 4, 2, 4, 2).permute(0, 1, 3, 2, 4).reshape(-1, 16, 4)
n_tokens, token_length = patches.shape[1], patches.shape[2]
print(patches.shape)
```

```
torch.Size([1797, 16, 4])
```

```
square = x[0].reshape(8, 8)[2:4, 2:4].flatten()
print(square, patches[0, 5])
```

```
tensor([0.9375, 0.1250, 0.7500, 0.0000]) tensor([0.9375, 0.1250, 0.7500, 0.0000])
```

```
# Stop the render if the prose below stops being true.
assert torch.equal(square, patches[0, 5])
```

The 8 by 8 grid holds 4 by 4 squares, so each digit becomes a sequence of 16 vectors, each holding the 4 pixels of one square. The second print checks that the sixth vector holds the square in rows 3 and 4 and columns 3 and 4, since the squares run along each row of squares before moving down. `reshape` and `permute` group the rows and columns into squares, and no pixel moves from one digit to another.

Definition 5.1 (Vision Transformer (ViT))**.** A **vision Transformer**, or **ViT**, cuts an image into square patches, maps each patch to a vector with one linear layer, adds a learned positional encoding, and passes the sequence through Transformer blocks [11, Sec. 3.1]. A classifier then reads one vector summarizing the sequence.

Example 5.1 (A small ViT for the digits)**.**

```
class TinyViT(nn.Module):
    def __init__(self, n_patches=16, patch_size=4, width=32):
        super().__init__()
        self.embed = nn.Linear(patch_size, width)
        self.position = nn.Parameter(torch.zeros(n_patches, width))
        layer = nn.TransformerEncoderLayer(
            width, nhead=4, dim_feedforward=64, dropout=0.0,
            batch_first=True, norm_first=True)
        self.blocks = nn.TransformerEncoder(
            layer, num_layers=2, enable_nested_tensor=False)
        self.head = nn.Linear(width, 10)

    def forward(self, patch_seq):
        hidden = self.blocks(self.embed(patch_seq) + self.position)
        return self.head(hidden.mean(dim=1))


_ = torch.manual_seed(0)
vit = TinyViT()
optimizer = torch.optim.Adam(vit.parameters(), lr=3e-3)
shuffler = torch.Generator().manual_seed(0)
for epoch in range(30):
    for batch in torch.randperm(n_train, generator=shuffler).split(50):
        rows = train_idx[batch]
        optimizer.zero_grad()
        loss_fn(vit(patches[rows]), y[rows]).backward()
        optimizer.step()
_ = vit.eval()
vit_count = sum(p.numel() for p in vit.parameters())
vit_acc = accuracy(vit, patches[val_idx], y[val_idx])
print(vit_count, vit_acc)
```

```
18090 0.9413735270500183
```

`TinyViT` is a vision Transformer with 2 Transformer blocks of width[a] 32, 18,090 parameters[b] in all. It averages the block outputs over the sequence before its last linear layer, rather than reading a special extra vector as Dosovitskiy et al. [11] do. Trained for 30 epochs[c] on the 1,000 training digits, it classifies 94% of the validation digits correctly. Dosovitskiy et al. [11, Sec. 4.3] find that ResNets, convolutional networks, do better than ViTs when both are first trained on small sets of labelled images, and ViTs do better after first training on the 303 million labelled images of JFT-300M.

[a] https://morrison-lab.github.io/lds/chapters/neural-nets.html#def-width
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[c] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-epoch

5.2 Further reading

Read

Brown et al. [4] introduced few-shot prompting with GPT-3, Hu et al. [7] LoRA, Radford et al. [10] Whisper, and Dosovitskiy et al. [11] the vision Transformer.

6 References

[1] E. Alpaydin and C. Kaynak, "Optical recognition of handwritten digits." UCI Machine Learning Repository, 1998. doi: 10.24432/C50P49[7].

[2] OpenAI and Hugging Face, "Openai-community/gpt2." Hugging Face model card, 2026. Available: https://huggingface.co/openai-community/gpt2

[3] Hugging Face, "Modeling_gpt2.py." Hugging Face Transformers source code, 2026. Available: https://github.com/huggingface/transformers/blob/main/src/transformers/models/gpt2/modeling_gpt2.py

[4] T. B. Brown *et al.*, "Language models are few-shot learners." 2020. Available: https://arxiv.org/abs/2005.14165

[5] J. Hoffmann *et al.*, "Training compute-optimal large language models." 2022. Available: https://arxiv.org/abs/2203.15556

[6] J. Kaplan *et al.*, "Scaling laws for neural language models." 2020. Available: https://arxiv.org/abs/2001.08361

[7] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," in *International conference on learning representations*, 2022. Available: https://arxiv.org/abs/2106.09685

[8] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback." 2022. Available: https://arxiv.org/abs/2203.02155

[9] Z. Jackson *et al.*, "Free spoken digit dataset (FSDD)." GitHub repository, release v1.0.10, 2018. doi: 10.5281/zenodo.1342401[8].

[10] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proceedings of the 40th international conference on machine learning*, in Proceedings of machine learning research, vol. 202. 2023, pp. 28492–28518. Available: https://proceedings.mlr.press/v202/radford23a.html

[11] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International conference on learning representations*, 2021. Available: https://arxiv.org/abs/2010.11929

[7] https://doi.org/10.24432/C50P49
[8] https://doi.org/10.5281/zenodo.1342401